BirdCLEF+ 2026: Bird Sound Recognition

Laboratory Work Report

Edvard Student, Matvii Shumylovych, Andrii Kravchuk

Kaggle Competition: `birdclef-2026`

1 Introduction and Competition Overview

BirdCLEF+ 2026 is a Kaggle competition where the goal is to automatically recognise bird species from audio recordings. The input is a long soundscape recording, which is split into 5-second segments. For each segment the model must predict the probability that each of 234 bird species can be heard in it. This is a multi-label classification problem because more than one species can be present at the same time.

One important rule of this competition is that models must run **on CPU only and without an internet connection** during evaluation. This means we had to think about how fast and how large the model is, not just how accurate it is. The main evaluation metric is padcROC, which is the macro-averaged ROC-AUC score with padding for species that have no positive examples in the test set.

The dataset includes:

- Short labelled audio clips in `.ogg` format, one folder per species
- Long unlabelled soundscape recordings
- Test soundscapes used for the final submission
- A `taxonomy.csv` file with all 234 species labels

2 Analysis of Existing Solutions

Before writing any code, we looked at what the top teams did in BirdCLEF 2025, which is basically the same competition from the previous year. This helped us understand which approaches are worth trying.

The main things we learned from the 2025 solutions:

- The most common approach is to convert audio into a **log-Mel spectrogram** and then use a CNN image classifier on it. This works well because spectrograms look like images.
- Everyone uses **5-second segments**, which matches how the competition evaluates predictions.

- A **SED (Sound Event Detection) head** works better than simple global average pooling because it keeps the time information and can learn when a bird is singing in the segment.
- Augmentations like SpecAugment, Mixup, and adding background noise help the model generalise from short clean clips to long noisy soundscapes.
- ONNX or OpenVINO export is needed to make inference fast enough on CPU without a timeout.
- Pseudo-labelling on unlabelled soundscapes and ensembling multiple models give the best scores in top solutions.

Based on this, we decided to build three different models so we can compare them and see which approach works best for this task.

3 Exploratory Data Analysis

3.1 Dataset Inventory and Class Distribution

We loaded the training data from `train.csv` and species names from `taxonomy.csv`. There are **234 unique bird species** in total. The dataset has 35,549 geo-tagged recordings. The class distribution is very unbalanced, as shown in Figures 1 and 2: the most common species have up to 500 recordings each, while the rarest ones have fewer than 5. We set `RARE_THRESH = 20` to mark rare species and handle them with weighted sampling during training.

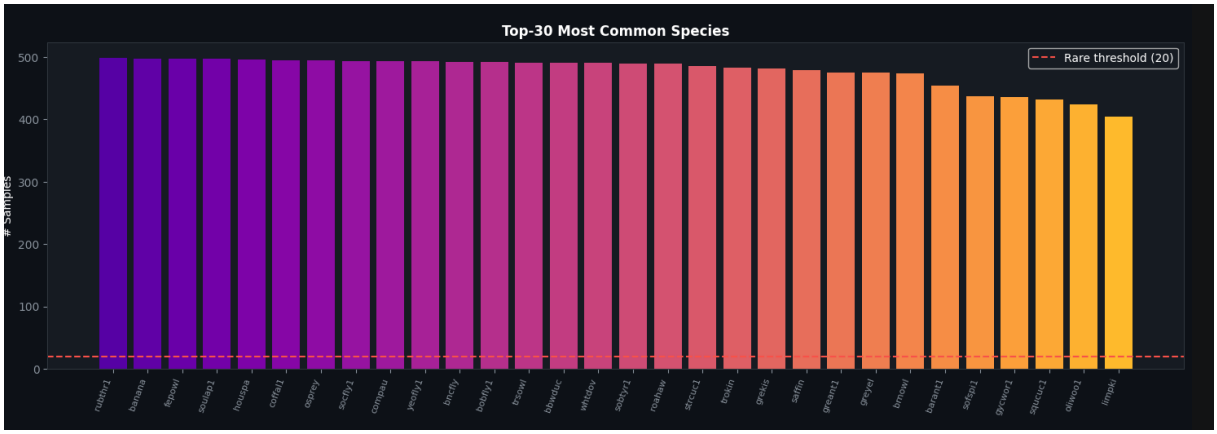

Figure 1: Top-30 most common species. The most frequent species have up to 500 training clips. The dashed red line marks the rare species threshold (20 samples).

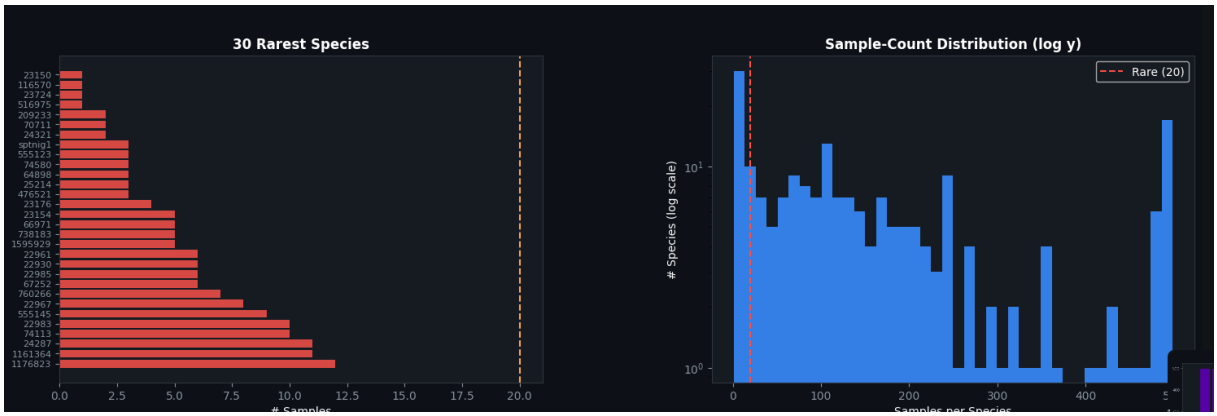

Figure 2: Left: the 30 rarest species, most having fewer than 5 samples. Right: the full sample-count distribution on a log scale, showing the long-tail nature of the dataset.

3.2 Audio Quality and Metadata Analysis

We checked the recording quality ratings, secondary label distribution, and geographic coverage of the dataset. Results are shown in Figure 3.

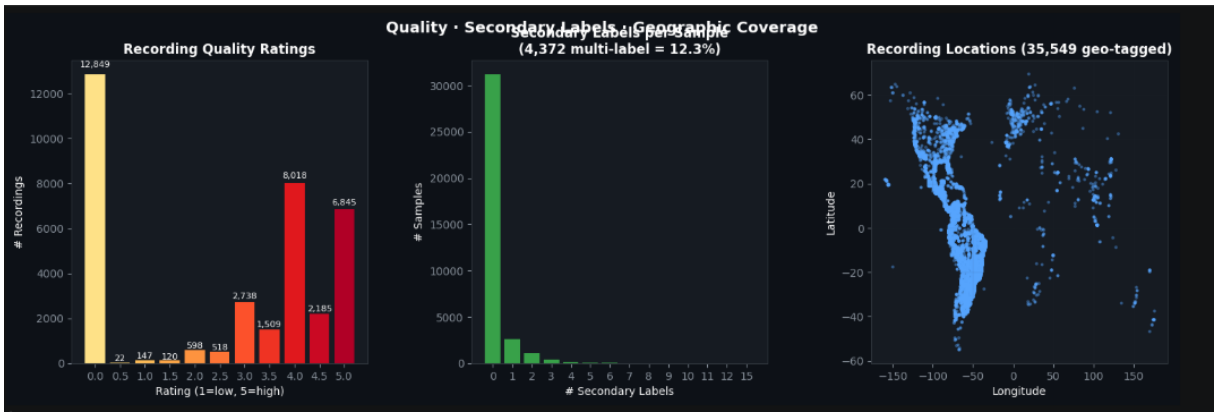

Figure 3: Left: most recordings have a quality rating of 4 or 5 (8,018 and 6,845 recordings respectively), confirming the dataset is generally high quality. Centre: 12.3% of samples have at least one secondary label, meaning multiple species are present. Right: recordings are geo-tagged across the Americas.

Key observations:

- Most recordings have high quality ratings (4–5), so the training data is generally clean.
- 4,372 recordings (12.3%) have secondary labels, confirming the multi-label nature of the problem.
- The recordings are spread across North and South America.

3.3 Mel Spectrogram Visualisation

We converted audio clips to log-Mel spectrograms and visually examined both individual clips and a grid of 12 different species. The parameters are shown in Table 1.

Table 1: Mel spectrogram parameters (same for all three models)

Parameter	**Value**
Sample rate	32,000 Hz
Segment length	5.0 s
FFT size (N_FFT)	1,024
Hop length	320
Mel bins (N_MELS)	128
Frequency range	20 – 16,000 Hz
Amplitude scale	Log (top_dB = 80)
Normalisation	Per-segment z-score

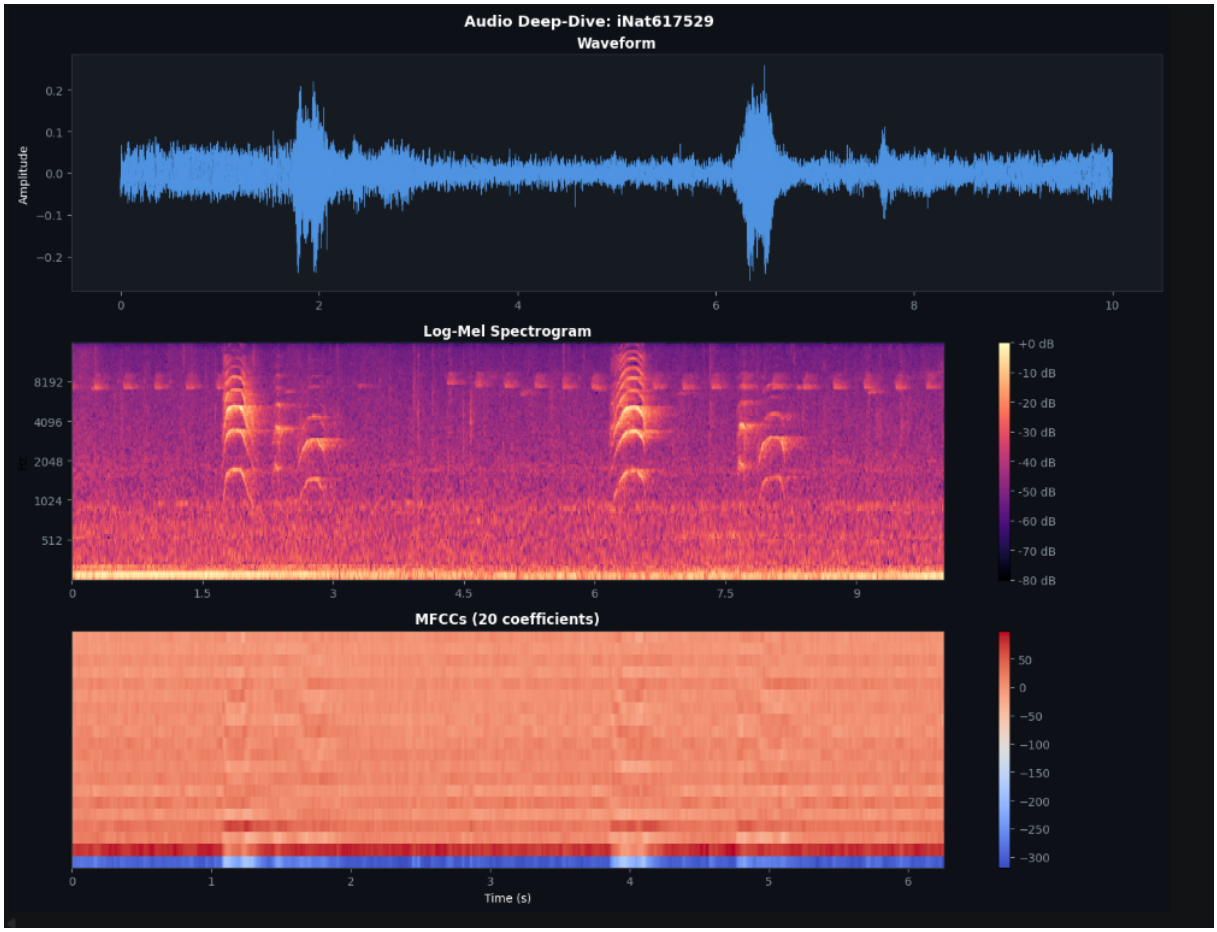

Figure 4: Audio deep-dive for one sample clip (iNat617529): raw waveform (top), log-Mel spectrogram (middle), and MFCC features (bottom). The bird vocalisations are clearly visible as bright arched patterns in the spectrogram around 1,000–4,000 Hz.

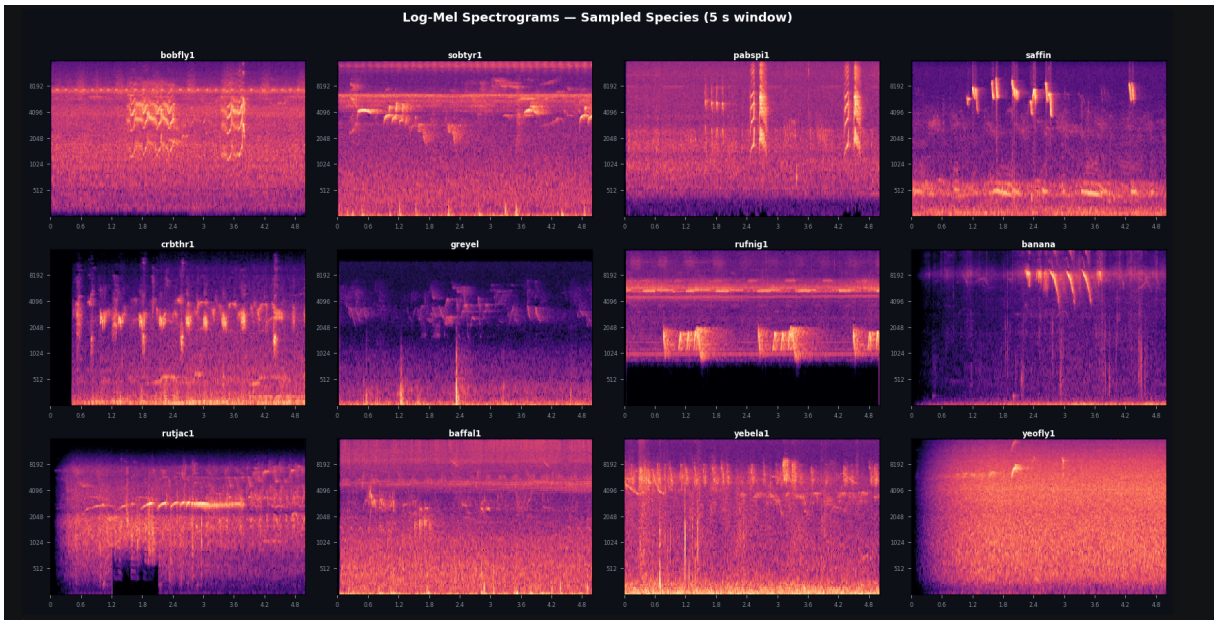

Figure 5: Log-Mel spectrograms for 12 randomly sampled species in a 5-second window. Each species has a clearly different visual pattern, which confirms that spectrograms are a useful input representation for bird classification.

3.4 Augmentation Visualisation

We also visualised the effect of different augmentation methods on the spectrogram, as shown in Figure 6. This confirms that the augmentations modify the spectrogram in a realistic way without destroying the bird call patterns.

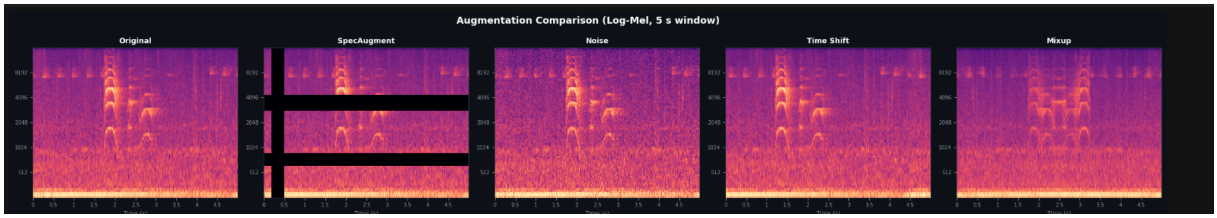

Figure 6: Comparison of augmentation methods on a 5-second log-Mel spectrogram: Original, SpecAugment (black frequency and time masks), Noise (background noise added), Time Shift, and Mixup. All augmentations preserve the general structure of the bird call and could be useful during training.

3.5 Cross-Validation Strategy

We used **StratifiedKFold** with $k = 5$ folds, splitting by `primary_label` so that every species appears in every fold. Figure 7 shows that the split is very balanced: each fold has approximately 7,000 validation samples and around 200 species are represented in every fold.

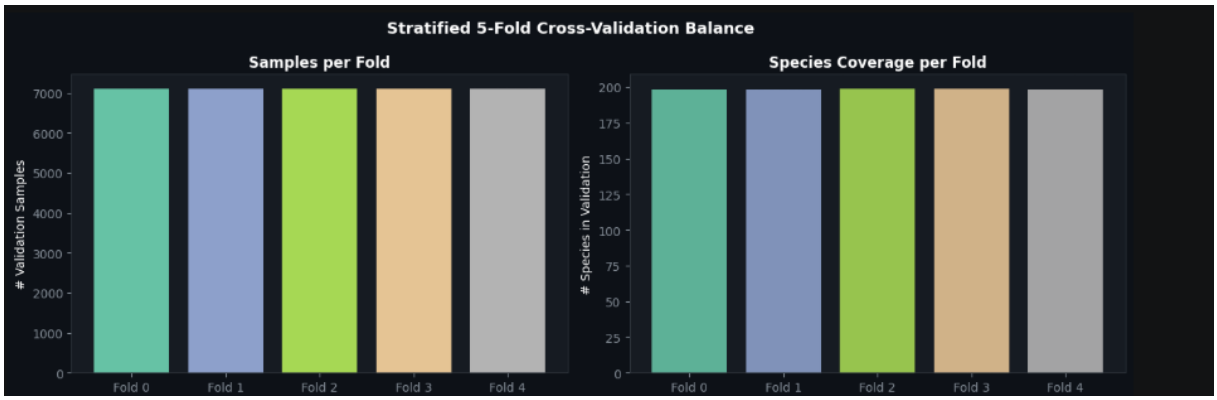

Figure 7: Stratified 5-fold cross-validation balance. Left: each fold has approximately 7,000 validation samples. Right: every fold covers around 200 species, confirming that the stratification works correctly.

Figure 8 shows how rare species are distributed across folds. Even the rarest species appear in multiple folds, which allows us to evaluate model performance on them.

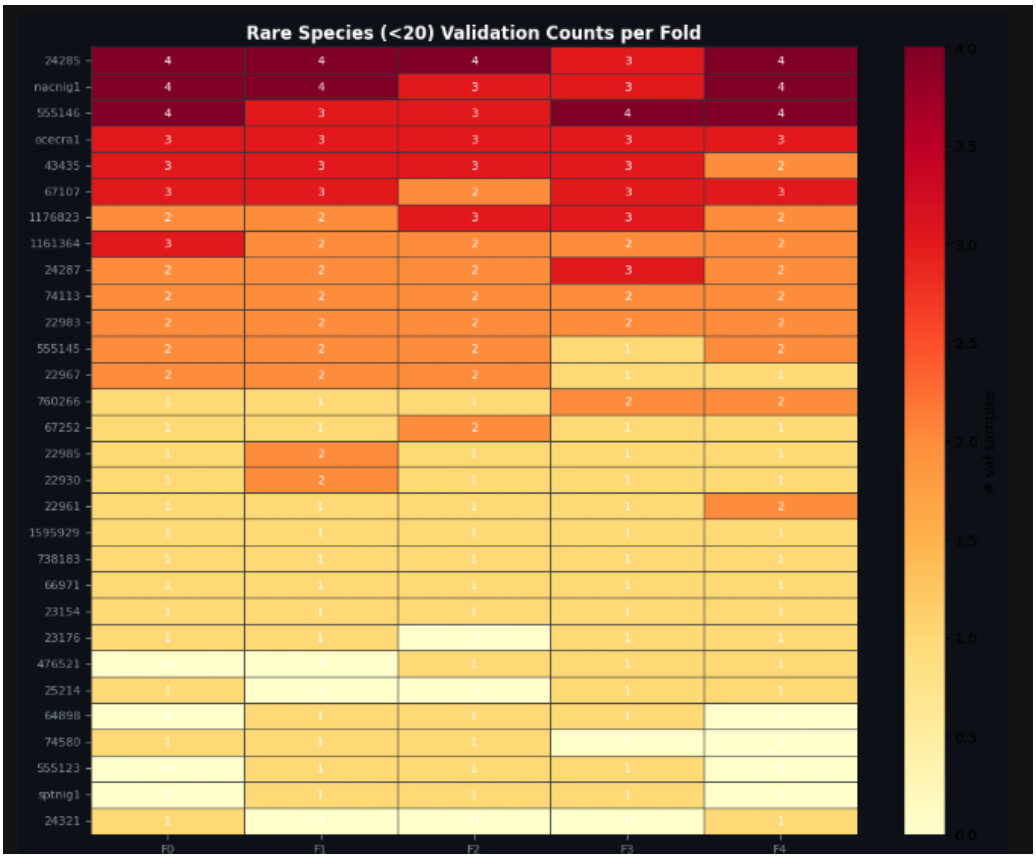

Figure 8: Validation counts for rare species (<20 total samples) across all 5 folds. The stratification ensures that even very rare species appear in each fold at least once or twice.

We used fold 4 as the validation set for all three models to keep the comparison fair. One thing worth noting is that validation AUC on short clean clips is usually a bit higher than the leaderboard score, because the test data consists of noisier long soundscapes.

4 Validation Framework

4.1 Validation Dataset

The validation set is fold 4 from the StratifiedKFold split. Each audio clip is loaded, converted to a mel spectrogram, and passed through the model to get predicted probabilities. We use a standard DataLoader with batch size 32 and 2 workers. To avoid noisy metric estimates for very rare species, we only include species that have at least `MIN_POSITIVES = 5` positive examples in the validation fold.

4.2 Evaluation Metrics

We compute three metrics for each model:

1. **padcROC** - the main competition metric. ROC-AUC is computed per species and then averaged across all species. Species with no positive examples in the validation set get a default score of 0.5 (random chance).
2. **Macro AUC** - standard macro-averaged ROC-AUC, only counting species that have at least 5 positive examples.
3. **mAP** - Mean Average Precision. This metric cares more about precision and is more strict than AUC, so it is a good extra signal for model quality.

4.3 Proposed Additional Metrics

Besides the competition metric, we also suggest these extra metrics that could be useful:

- **Recall@K**: checks whether the correct species is in the top-K predictions. This is useful in practice when we just need a short ranked list.
- **Per-class F1 at threshold 0.5**: a simple binary metric that shows real-world classification performance when a decision threshold is applied.
- **Soundscape-level AUC**: evaluates predictions aggregated over full soundscape files instead of individual 5-second segments, which is closer to the actual competition evaluation.

5 Model Development

We implemented three independent models, each trained from scratch (no pre-trained weights used).

5.1 Common Preprocessing Pipeline

All three models use the same audio preprocessing steps:

1. Load audio with `torchaudio`, convert stereo to mono.
2. Resample to 32 kHz if needed.
3. Take a 5-second segment (random crop during training, centre crop during validation).

4. Zero-pad segments that are shorter than 5 seconds.
5. Compute log-Mel spectrogram (parameters in Table 1).
6. Normalise: subtract mean and divide by standard deviation for each segment.

5.2 Training Augmentations (M1 and M3)

During training we used **WeightedRandomSampler** to oversample rare species (those with fewer than 20 clips), and we used a random start position when cutting the 5-second segment from each clip.

5.3 Loss Function

All three models are trained with **Focal Loss** for multi-label classification:

$$\mathcal{L}_{\text{focal}} = -\sum_{c=1}^{C} \left[\alpha(1-p_c)^{\gamma}\, y_c \log p_c + (1-\alpha)\, p_c^{\gamma}\, (1-y_c) \log(1-p_c)\right]$$

with $\alpha = 0.25$ and $\gamma = 2.0$. Focal Loss gives less weight to easy negative examples, which helps when the dataset is very imbalanced.

5.4 Model 1: EfficientNetV2-S with SED Head (M1)

We use the `efficientnetv2_s` backbone from `timm` with single-channel input. After the backbone produces feature maps, we average over the frequency axis to get a time sequence, apply dropout ($p = 0.3$), and pass through two parallel linear layers: one for clip-level scores and one for attention weights. The final output is:

$$\text{out} = \sum_{t} \sigma(\text{clip\_logit}_t) \cdot \text{softmax}(\text{att\_weight}_t)$$

This SED head lets the model decide which time steps matter most.

Table 2: M1 training configuration

Parameter	**Value**
Backbone	EfficientNetV2-S
Epochs	12
Batch size	32
Optimiser	AdamW, LR = 10^{-3}, WD = 10^{-4}
Scheduler	CosineAnnealingLR

5.5 Model 2: Audio Spectrogram Transformer (M2)

We built an AST from scratch: the spectrogram is split into 16×16 patches and projected to 384 dimensions, then processed by 12 Transformer blocks (6 heads, GELU, MLP ratio 4.0). A learnable CLS token is used as the global representation, passed through a two-layer MLP head with sigmoid output.

Table 3: M2 training configuration

Parameter	**Value**
Architecture	AST (built from scratch)
d_{model} / layers / heads	384 / 12 / 6
Epochs	15
Batch size	32
Optimiser	AdamW, LR $= 5 \times 10^{-4}$, WD $= 10^{-4}$

5.6 Model 3: ResNet-34 with SED Head (M3)

Same SED attention head as M1, but with a lighter `resnet34` backbone (single-channel, no pre-training). Because ResNet-34 is smaller, we could use a larger batch size.

Table 4: M3 training configuration

Parameter	**Value**
Backbone	ResNet-34
Epochs	12
Batch size	64
Optimiser	AdamW, LR $= 3 \times 10^{-4}$, WD $= 10^{-4}$
Scheduler	CosineAnnealingLR

6 CPU Inference Pipeline

Because the competition requires no internet access and CPU-only inference, we separated training and inference into different notebooks. Running full training followed by test inference in one Kaggle kernel caused a timeout error. We trained on GPU kernels, saved weights to a Kaggle dataset, and loaded them in a separate inference kernel.

For M1 and M3 we also exported to ONNX format to speed up CPU inference. The ONNX Runtime wheels were pre-downloaded and uploaded as a Kaggle dataset (`birdclef-2026-download-whe` because pip install is not possible without internet.

The inference steps are: load the soundscape, split into 5-second segments, compute mel spectrogram, run the model, map each prediction to the correct `row_id`, and save `submission.csv`.

7 Results and Leaderboard Scores

7.1 Validation Results

Figure 9 shows the validation scores for all three models across the three metrics.

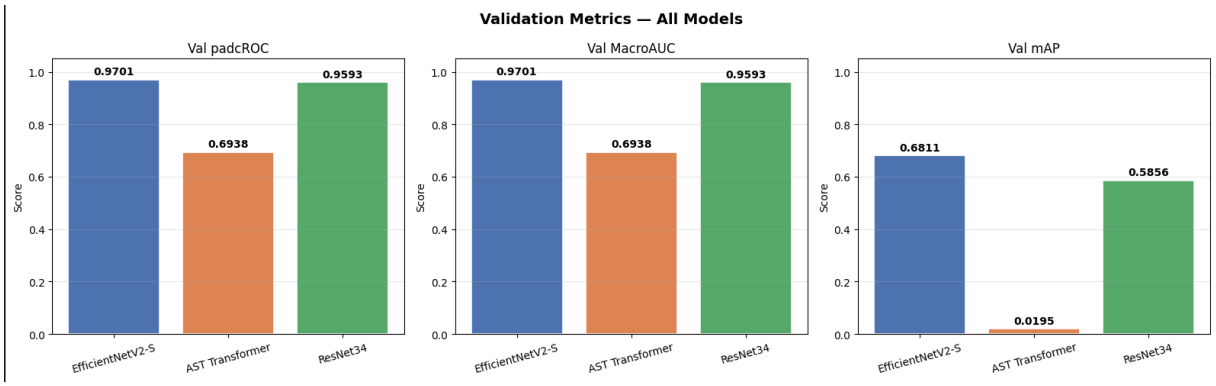

Figure 9: Validation metrics for all three models. EfficientNetV2-S and ResNet-34 both score above 0.95 on padcROC and MacroAUC. The AST Transformer scores only 0.6938 on AUC and 0.019 on mAP, showing that its predictions have poor precision.

Table 5: Exact validation and leaderboard results for all three models

Model	Val padcROC	Val MacroAUC	Val mAP	LB Score
M1 - EfficientNetV2-S + SED	0.9701	0.9701	0.6811	**0.748**
M3 - ResNet-34 + SED	0.9593	0.9593	0.5856	0.721
M2 - AST (from scratch)	0.6938	0.6938	0.0195	0.544

7.2 Per-Class AUC Distribution

Figure 10 shows how the AUC is distributed across individual species. EfficientNetV2-S achieves a mean AUC of 0.970 with most classes scoring above 0.95. ResNet-34 is close with a mean of 0.959. The AST has a mean of only 0.694, with a wide spread showing many species are poorly classified.

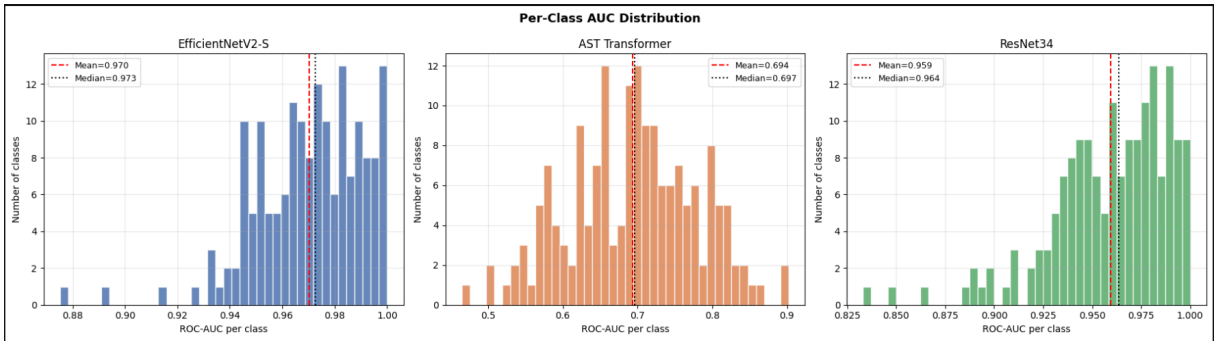

Figure 10: Per-class AUC distribution for all three models. EfficientNetV2-S (mean=0.970, median=0.973) and ResNet-34 (mean=0.959, median=0.964) both have most species scoring above 0.93. The AST Transformer (mean=0.694, median=0.697) has a much lower and wider spread, meaning it struggles on many individual species.

7.3 Validation-to-Leaderboard Correlation

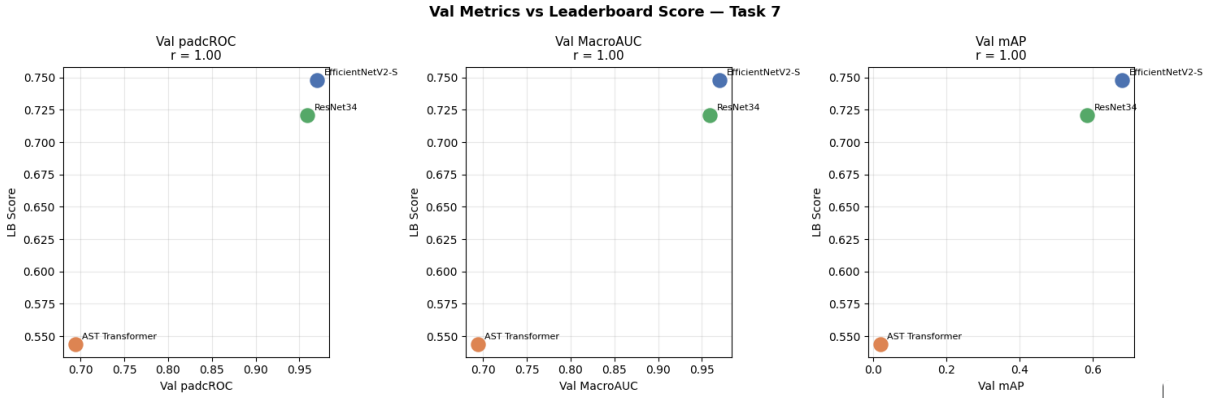

Figure 11: Validation metrics vs leaderboard score for all three models. Pearson $r = 1.00$ for all three metrics, meaning the model ranking on validation perfectly matches the ranking on the leaderboard.

As Figure 11 shows, the Pearson correlation between each validation metric and the leaderboard score is $r = 1.00$. This tells us that our fold-4 validation set is a reliable proxy for the leaderboard, and that improving any of the three validation metrics should translate into a better leaderboard score. The mAP metric has a very different scale from AUC (especially for the AST, where mAP is near zero), but the rank-ordering is still perfect.

8 Comparison with BirdCLEF 2025

Table 6: Comparison of BirdCLEF 2025 vs BirdCLEF 2026

Aspect	**BirdCLEF 2025**	**BirdCLEF 2026**
Number of species	∼206	234
Inference hardware	CPU	CPU
Internet during test	No	No
Primary metric	padcROC	padcROC
Top LB scores	0.93	0.951
Top architectures	EfficientNet, ConvNeXt + SED	EfficientNet, ResNet + SED
ONNX/OpenVINO	Widely used	Necessary
Pseudo-labelling	Common in top solutions	Applicable

The competitions are very similar. The number of species grew from about 206 to 234, making the problem a bit harder. Our best score (0.748) is competitive for this stage. Top 2025 solutions reached 0.93 by using ensembles, pseudo-labelling, and pre-training on older competition data - things we have not done yet. Our AST result also confirms what 2025 solutions showed: transformer models need pre-training to compete with CNNs on datasets of this size.

9 Discussion

9.1 Why EfficientNetV2-S Beats ResNet-34

EfficientNetV2-S uses a more modern design with Fused-MBConv blocks. With the same SED head, the difference in leaderboard score (0.748 vs 0.721) and validation padcROC (0.9701 vs 0.9593) mostly comes from the backbone being better at extracting features from the spectrogram.

9.2 Why the AST Underperforms

Transformer models need a lot of data or pre-training to work well. The per-class AUC distribution (Figure 10) makes this very clear: the AST has a mean AUC of only 0.694 and a mAP of 0.019, while both CNNs score above 0.95 on AUC. The model's ranked predictions have almost no precision, which means it cannot reliably identify which species are present even when it sometimes assigns high probability to the right one.

9.3 Limitations and What Could Be Improved

- **No pseudo-labelling**: using the unlabelled soundscapes to generate extra training data could help a lot.
- **No pre-training**: loading pre-trained weights would significantly improve all models.
- **No ensembling**: combining M1 and M3 predictions is an easy next step.
- **Augmentations not fully used during training**: SpecAugment and Mixup (shown in Figure 6) were visualised but not applied during training of all models.
- **AST pre-training**: an AST initialised from an AudioSet checkpoint would be a much stronger M2 baseline.

10 Conclusion

We built a full pipeline for BirdCLEF+ 2026: exploratory data analysis, cross-validation, three different models, ONNX-optimised CPU inference, and leaderboard submissions.

Our best model - EfficientNetV2-S with a SED attention head, 12 epochs, AdamW, Focal Loss - got a leaderboard score of **0.748** and a validation padcROC of **0.9701** (per-class mean AUC = 0.970). ResNet-34 with the same SED head scored 0.721 on the leaderboard (padcROC = 0.9593, mean AUC = 0.959). The AST from scratch scored 0.544 on the leaderboard (padcROC = 0.6938, mAP = 0.019), confirming that CNN-based models are much more practical without pre-training.

All three validation metrics had a perfect rank-correlation ($r = 1.00$) with the leaderboard, confirming that our validation setup is reliable. The main infrastructure challenge - CPU-only, no-internet inference - was solved through ONNX export and pre-packaged dependencies on Kaggle.

A File Structure

- `eda.ipynb` - Exploratory Data Analysis
- `validation.ipynb` - Model validation and metric computation
- `training_1.ipynb` - Training M1 (EfficientNetV2-S + SED)
- `training_resnet34.ipynb` - Training M3 (ResNet-34 + SED)
- `training_3.ipynb` - Training M2 (AST)
- `resnet34mod3.ipynb` - ResNet-34 model definition
- `test_1.ipynb` - Inference for M1 (CPU, offline)
- `test_3.py` - Inference for M2 (CPU, offline)

B Hyperparameter Summary

Table 7: All model hyperparameters and results

Parameter	**M1 (EffNetV2-S)**	**M2 (AST)**	**M3 (ResNet-34)**
Backbone	EfficientNetV2-S	Custom AST	ResNet-34
Pre-training	None	None	None
Epochs	12	15	12
Batch size	32	32	64
Learning rate	10^{-3}	5×10^{-4}	3×10^{-4}
Loss	Focal	Focal	Focal
Head	SED attention	MLP	SED attention
Val padcROC	0.9701	0.6938	0.9593
Val mAP	0.6811	0.0195	0.5856
Mean per-class AUC	0.970	0.694	0.959
LB Score	**0.748**	0.544	0.721